Pokhara University

Gandaki College of Engineering and Science

Lamachaur, Kaski

Lab 3: Implementation of Software Testing – Test Case Generation, Jest, JUnit, Selenium, and Load Testing

Submitted By:

Rohan Baral

Roll no. 32

Submitted To:

Er. Rajendra Bahadur Thapa

OBJECTIVE :

This lab focuses on implementing software testing techniques covering test case generation, and hands-on use of Jest, JUnit, Selenium, and Artillery for load testing.

AIM:

To explore and apply a range of software testing approaches, including unit testing with Jest and JUnit, browser automation testing using Selenium, and load testing using Artillery.

THEORY :

Software testing is the practice of evaluating an application in order to uncover defects and confirm that it satisfies its requirements. Within Agile development, testing is woven into every sprint rather than being deferred to a final phase.

- Test Case Generation: pinpointing the inputs, expected outcomes, and conditions needed to confirm correct software behaviour.
- Jest: a JavaScript testing framework developed by Meta, commonly applied to unit and integration testing for Node.js and React applications.
- JUnit: a Java unit testing framework relying on annotations such as @Test and @BeforeEach to define and execute test cases.
- Selenium: a browser automation tool capable of simulating user actions, including clicks, form submissions, and page navigation, to test web interfaces.
- Artillery: a contemporary load testing tool that generates simulated HTTP traffic to gauge application performance and surface bottlenecks under heavy load.

OBJECTIVES:

- Write and execute unit tests for a JavaScript function using Jest.
- Write and execute unit tests for a Java method using JUnit.
- Automate an interaction within a browser using Selenium.
- Execute a basic load test with Artillery and interpret its results.

ALGORITHM:

1. Write a JavaScript function together with a corresponding Jest test file.
2. Execute the Jest test suite and confirm the results.
3. Write a Java method together with a corresponding JUnit test class.
4. Execute the JUnit tests either through Maven or directly from an IDE.
5. Write a Selenium script that launches a browser, navigates to a URL, and verifies its content.
6. Write an Artillery YAML configuration file to simulate concurrent user traffic.
7. Execute Artillery and examine the resulting summary report.

CODE :

Jest – JavaScript unit test:

```
// math.js

function add(a, b) { return a + b; }

module.exports = { add };



// math.test.js

const { add } = require('./math');

test('adds 2 + 3 to equal 5', () => {

    expect(add(2, 3)).toBe(5);

});
```

JUnit – Java unit test:

```
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;



public class CalculatorTest {

    @Test

    public void testAdd() {

        assertEquals(5, new Calculator().add(2, 3));

    }

}
```

Selenium – Browser automation (Python):

```
from selenium import webdriver

from selenium.webdriver.common.by import By



driver = webdriver.Chrome()

driver.get('https://www.google.com')

assert 'Google' in driver.title

print('Test Passed:', driver.title)

driver.quit()
```

Artillery – Load test config (load_test.yml):

```
config:

  target: 'http://localhost:3000'

  phases:

    - duration: 30

      arrivalRate: 10

scenarios:

  - flow:

      - get:

          url: '/api/users'
```

OUTPUT :

Jest:

```
PASS math.test.js

✓  adds 2 + 3 to equal 5

Tests: 1 passed
```

JUnit:

```
Tests run: 1, Failures: 0, Errors: 0
```

Artillery:

```
Requests completed: 300
Response time p95: 120ms
HTTP 200: 300
```

RESULT :

Every testing framework was implemented and exercised successfully: Jest and JUnit confirmed the correctness of unit-level logic, Selenium handled automated browser interactions, and Artillery evaluated API performance under simulated load.

CONCLUSION :

Testing stands as a central Agile practice for maintaining software quality throughout development. Automated tests shorten feedback loops and support continuous integration pipelines.